



1. Сначала определяю, что именно мы планируем

Сейчас у нас есть эпик «Самостоятельный возврат оплаченного заказа». Я не начинаю сразу придумывать story points. Сначала мне нужно понять границы работы.

Бизнес хочет, чтобы пользователь мог открыть уже полученный заказ, выбрать позиции для возврата, отправить заявку, видеть её состояние, а после подтверждения получить возврат денежных средств. Дополнительно должна существовать минимальная возможность для администратора подтвердить или отклонить заявку.

При этом у нас уже есть важные ограничения. Возврат разрешён только в течение 14 дней после получения заказа, возможен частичный возврат, нельзя вернуть больше единиц товара, чем было куплено, нельзя повторно вернуть уже полностью возвращённую позицию. Сумму нужно рассчитывать на основании фактически оплаченной стоимости с учётом скидки.

Я сразу фиксирую ещё один важный вывод: весь эпик не обязан попасть в один спринт. Ёмкость команды ограничена 24 story points, поэтому наша задача — выбрать законченный инкремент, а не искусственно занижить оценки, чтобы поместить всё.

Для демонстрации я могу обратиться к AI-агенту с таким промптом:

Ты участвуешь в планировании спринта команды разработки интернет-магазина.

Есть эпик: "Самостоятельный возврат оплаченного заказа".

Не составляй окончательный план спринта.

Сначала проанализируй описание эпика и выдели:

- основную пользовательскую ценность;
- обязательные части первого рабочего релиза;
- функции, которые можно отложить;
- места, где требования пока недостаточно определены.

Не придумывай отсутствующие бизнес-правила. Если данных недостаточно, укажи неопределённость явно.

Ответ агента можно рассмотреть, но дальше работаю с исходными требованиями самостоятельно.

2. Формулирую цель спринта

Теперь мне нужна не просто коллекция задач, а цель спринта.

Я формулирую её так:

«Подготовить минимальный рабочий сценарий самостоятельного создания заявки на возврат товара с расчётом суммы, административным решением по заявке и отображением её состояния пользователю».

Обратите внимание, я пока не включаю в формулировку гарантированный автоматический возврат денег через внешний платёжный сервис.

Причина техническая. Мы знаем, что метод `PaymentClient.refund()` существует, но команда ещё не использовала его в production. Дополнительно неизвестно, безопасно ли повторять запрос после timeout. Значит, вокруг этого участка есть существенная неопределённость.

Я не хочу строить весь Sprint Goal вокруг компонента, поведение которого команда ещё не исследовала.

При этом исследование платёжного возврата обязательно должно попасть в спринт, потому что без него мы не сможем нормально подготовить следующий этап.

3. Отделяю неопределённость от обычной разработки

Перед декомпозицией я отдельно смотрю на вопросы, на которые команда пока не знает ответа.

У нас нет утверждённого поведения для старых заказов без `delivered_at`. Не определена retry-стратегия для `PaymentClient.refund()` после timeout. Не утверждён окончательный набор статусов. Не решено, нужен ли комментарий администратора при отклонении и сможет ли пользователь отменять заявку.

Я не буду делать вид, что все эти вопросы одинаково блокируют разработку.

Например, возможность самостоятельной отмены заявки можно не включать в первый релиз. Основной пользовательский сценарий без неё остаётся рабочим.

Обязательный комментарий администратора тоже не блокирует создание базового механизма возврата. Если бизнес позднее решит добавить комментарий, это можно расширить отдельной задачей.

А вот проблема `delivered_at` влияет непосредственно на правило «14 дней после получения». Здесь без решения мы не можем полностью определить eligibility возврата для старых заказов.

Поведение `refund()` при timeout тоже существенно, потому что неправильный автоматический повтор потенциально может привести к повторной финансовой операции.

Поэтому именно эти две неопределённости я рассматриваю как наиболее важные для текущего спринта.

4. Провожу первичную декомпозицию эпика

Теперь я начинаю превращать эпик в задачи.

Первая задача — определить и реализовать модель заявки на возврат. Нам нужна отдельная сущность возврата и связь с заказом и возвращаемыми позициями.

Вторая задача — реализовать серверную проверку возможности возврата и расчёт суммы. Здесь находятся бизнес-правила по сроку, количеству, предыдущим возвратам и скидкам.

Третья задача — создать API для отправки заявки пользователем.

Четвёртая задача — дать пользователю возможность увидеть существующую заявку и её статус.

Пятая задача — сделать минимальное действие в административной части для подтверждения или отклонения заявки.

Шестая задача — добавить пользовательский интерфейс создания возврата.

Седьмая задача — добавить уведомление о создании заявки.

Отдельно появляется исследовательская задача по `PaymentClient.refund()`, потому что до реализации финансовой операции нам необходимо разобраться с timeout и безопасностью повторного запроса.

Для AI-агента здесь можно показать другой тип промпта:

Проведи предварительную декомпозицию эпика "Самостоятельный возврат оплаченного заказа".

Разбивай эпик на задачи, которые можно независимо оценивать и проверять.

Не оценивай задачи в story points.

Для каждой предлагаемой задачи объясни, какой конкретный результат должен быть получен после её завершения.

Отдельно укажи задачи, в которых слишком много неопределённости и сначала требуется исследование.

После ответа агента я сравниваю его вариант со своим. Если модель обнаружила пропущенную мной зависимость, это полезный сигнал. Но я не переношу её декомпозицию автоматически в backlog.

5. Проверяю, не получились ли задачи слишком большими

Теперь смотрю на полученную декомпозицию критически.

Например, формулировка «реализовать backend возвратов» для меня слишком большая. Внутри неё одновременно окажутся модель данных, правила eligibility, расчёт стоимости, API, статусы и административная обработка.

Такую задачу будет трудно оценивать и трудно считать завершённой.

Поэтому backend я разделяю.

Для модели данных ожидаемым результатом будет возможность хранить заявку на возврат, её статус и выбранные позиции.

Для расчётной логики результатом будет функция или сервис, который проверяет допустимость возврата и рассчитывает сумму.

Для API результатом будет возможность пользователя создать заявку через определённый endpoint.

Для административной операции результатом будет изменение статуса заявки сотрудником.

Теперь каждая задача имеет более конкретный результат.

6. Формирую рабочий набор задач

После декомпозиции у меня получается следующий предварительный backlog.

Задача	Что должно быть готово	Предварительная оценка
Модель данных возврата	Заявка, позиции возврата, количество, статус, связь с заказом	3 SP
Правила eligibility и расчёт суммы	Проверка 14 дней, количества, предыдущих возвратов и скидки	5 SP
API создания заявки	Пользователь может отправить корректную заявку	3 SP
API получения состояния возврата	Пользователь может получить статус своей заявки	2 SP
Минимальная административная обработка	Администратор может подтвердить или отклонить заявку	3 SP
Web-интерфейс создания возврата	Выбор позиций и количества, отправка заявки	5 SP
Отображение статуса на странице заказа	Пользователь видит состояние возврата	2 SP
Email после создания заявки	Пользователь получает уведомление	2 SP
Исследование <code>PaymentClient.refund()</code>	Понятен контракт, timeout и безопасная retry-стратегия	2 SP
Реализация возврата денег	Подтверждённая заявка инициирует возврат через PaymentClient	5 SP
Email после возврата денег	Уведомление после успешного refund	2 SP

Если сложить всё, я получаю 34 story points.

Это больше доступных 24.

Именно здесь появляется реальное планирование. Я не уменьшаю оценки искусственно. Мне нужно выбрать границу первого инкремента.

7. Объясняю, почему оценки именно такие

Теперь я проговариваю оценки, чтобы вы увидели, что story points не должны появляться случайно.

Модель данных я оцениваю в 3 SP. Здесь есть несколько связанных сущностей, но сама задача достаточно понятна и изолирована.

Бизнес-правила и расчёт суммы получают 5 SP. Причина не в количестве кода, а в количестве условий. Нужно учитывать срок, частичные возвраты, количество единиц, уже совершённые возвраты и фактически оплаченную стоимость с учётом скидки. Ошибка здесь может повлиять на деньги.

API создания заявки я оцениваю в 3 SP. Основная бизнес-логика должна находиться в уже выделенном сервисе, поэтому endpoint в основном проверяет входные данные и вызывает существующую логику.

Получение состояния — 2 SP, потому что это значительно проще создания и практически не содержит новой бизнес-логики.

Административная обработка — 3 SP. Мы не создаём новую сложную админ-панель, а минимально расширяем существующую.

Frontend создания возврата получает 5 SP, потому что здесь нужно отображать доступные позиции, выбрать количество, обрабатывать ограничения и связываться с новым API.

Отображение статуса я оцениваю в 2 SP, потому что основная страница заказа уже существует.

Исследование `PaymentClient.refund()` получает 2 SP не потому, что я знаю объём реализации, а потому, что ограничиваю исследование конкретным результатом. Команда должна получить ответ о контракте вызова, timeout и допустимой retry-стратегии.

Саму интеграцию refund я оцениваю отдельно в 5 SP. Она относится к финансовой операции и зависит от результата исследования.

8. Показываю пример оценки через AI, но не принимаю её автоматически

На этом этапе можно продемонстрировать агенту уже подготовленный backlog.

Промпт:

Перед тобой предварительная декомпозиция эпика возвратов интернет-магазина.

Используй относительную шкалу:

1, 2, 3, 5, 8, 13 story points.

Оцени каждую задачу не по количеству строк кода, а с учётом:
технической сложности;
числа зависимостей;
объёма тестирования;
риска ошибок;
неопределённости требований.

Для каждой оценки дай короткое обоснование.

Не составляй спринт и не меняй приоритеты задач.

Теперь я могу сравнить оценку AI со своей.

Если модель, например, поставила UI создания возврата 3 SP, а команда считает 5 SP, я не ищу «правильную цифру модели». Я выясняю причину расхождения.

Возможно, агент предполагает, что все компоненты интерфейса уже существуют. Возможно, команда знает, что форма выбора количества требует дополнительной работы.

Оценка полезна именно как повод для обсуждения.

9. Выявляю зависимости между задачами

Теперь я проверяю, что нельзя начинать независимо.

API создания заявки зависит от модели данных и основной бизнес-логики.

Frontend создания возврата зависит от согласованного API-контракта, хотя часть интерфейса можно начать раньше с mock-данными.

Отображение статуса зависит от возможности получить состояние возврата.

Административная обработка зависит от существования самой заявки и её статусов.

Интеграция с `PaymentClient.refund()` зависит от результата исследования внешнего клиента.

Я также учитываю доступность команды. Frontend-разработчик первые два дня занят старой задачей, поэтому нет смысла строить план так, будто UI начнёт разрабатываться в первый час спринта.

Специалист по платежам будет отсутствовать два дня в середине спринта. Поэтому исследование `PaymentClient.refund()` разумнее провести в начале, пока нужный разработчик доступен.

10. Отдельно решаю вопрос с `delivered_at`

Теперь у меня остаётся неопределённость старых заказов без `delivered_at`.

Я не хочу, чтобы разработчик самостоятельно придумал бизнес-правило.

Поэтому в рамках задачи по eligibility фиксирую ограничение: для новых корректных заказов используется `delivered_at`, а обработка исторических заказов должна быть уточнена у Product Owner до завершения задачи.

Если ответ Product Owner не будет получен вовремя, я не буду считать это основанием автоматически разрешить возврат.

В production финансовые и временные ограничения лучше не ослаблять догадкой.

Для демонстрации AI можно использовать такой промпт:

Проанализируй только неопределённости эпика возвратов.

Не предлагай самостоятельно бизнес-решения.

Раздели их по смыслу:

какие вопросы блокируют начало разработки;
какие блокируют только завершение конкретной задачи;
какие можно безопасно отложить после первого релиза.

Для каждого вывода объясни причину.

Это хороший пример использования агента не для написания решения, а для анализа риска.

11. Выбираю задачи текущего спринта

Теперь у меня есть ограничение:

24 story points

Я собираю первый рабочий инкремент.

В спринт беру модель данных — 3 SP.

Беру правила eligibility и расчёта суммы — 5 SP.

Беру API создания заявки — 3 SP.

Беру API получения состояния — 2 SP.

Беру минимальную административную обработку — 3 SP.

Беру Web-интерфейс создания возврата — 5 SP.

Беру исследование `PaymentClient.refund()` — 2 SP.

Получаю:

$3 + 5 + 3 + 2 + 3 + 5 + 2 = 23 \text{ SP}$

У нас остаётся один story point, но я не пытаюсь обязательно заполнить его.

Story points не являются валютой, которую нужно потратить до последней единицы.

23 SP при capacity 24 SP — нормальный план.

12. Что я сознательно не включаю в спринт

Я не включаю в основной commitment реализацию фактического возврата денег через

```
PaymentClient.refund()
```

.

Причина заключается в том, что эта задача зависит от исследования поведения API, особенно после timeout.

Я также не включаю отдельное email-уведомление после финансового возврата, потому что самого автоматического refund ещё нет.

Email после создания заявки тоже оставляю за пределами основного commitment. Он полезен, но базовый пользовательский сценарий остаётся работоспособным и без него.

Отображение специального статуса непосредственно на странице заказа также можно добавить следующим небольшим шагом, поскольку пользователь уже сможет получить состояние через соответствующую часть интерфейса возврата.

Так я сохраняю фокус на основном сценарии, а не пытаюсь закончить весь эпик за один спринт.

13. Планирую демонстрацию конца первой недели

Теперь вспоминаю дополнительное условие Product Owner: первый рабочий результат желательно показать уже через неделю.

К концу первой недели я хочу продемонстрировать сценарий без фактического движения денег.

Пользователь открывает подходящий заказ, выбирает позиции, система проверяет ограничения и рассчитывает сумму, после чего создаётся заявка со статусом `requested`.

Это уже законченный вертикальный фрагмент пользовательского сценария.

Для такой демонстрации нам нужны модель данных, основная логика, API создания и минимальная часть frontend.

Административную обработку можно закончить после первой демонстрации, но до конца спринта.

Параллельно в начале недели backend-разработчик, знакомый с платежами, выполняет исследование `PaymentClient.refund()` до своего отсутствия.

14. Распределяю работу логически, но не превращаю story points в часы сотрудников

Первый backend-разработчик начинает с исследования `PaymentClient.refund()` и помогает определить модель возврата, потому что хорошо знает существующий платёжный и заказный контекст.

Второй backend-разработчик может работать над моделью данных и затем над API, но я учитываю, что он недавно в проекте. Поэтому критическую финансовую интеграцию ему сейчас не передаю как полностью самостоятельную работу.

Frontend-разработчик подключается к интерфейсу после первых двух дней. До готовности backend endpoint можно работать с согласованным контрактом или mock-данными.

QA начинает готовить проверки бизнес-правил ещё до завершения всех компонентов, особенно сценарии частичного возврата, превышения количества и срока 14 дней.

Я не перевожу 23 story points в формулу вроде «один разработчик должен сделать 8 points». Story points относятся к относительной сложности задач команды, а не к индивидуальной производительности сотрудника.

15. Проверяю спринт с помощью AI как независимого критика

Теперь AI можно использовать ещё одним способом.

Я уже самостоятельно составил спринт.

Теперь прошу модель не планировать вместо меня, а искать слабые места моего решения:

Проведи критическую проверку готового Sprint Plan.

Capacity команды: 24 story points.

Выбранные задачи:

модель данных возврата – 3 SP;
правила eligibility и расчёт суммы – 5 SP;
API создания заявки – 3 SP;
API получения состояния – 2 SP;
минимальная административная обработка – 3 SP;
Web-интерфейс создания возврата – 5 SP;
исследование PaymentClient.refund() – 2 SP.

Итого: 23 SP.

Не составляй альтернативный спринт.

Найди только:
неучтённые зависимости;
возможные блокирующие неопределённости;
слишком крупные задачи;
риски, из-за которых план может оказаться нереалистичным.

Каждое замечание обоснуй исходными условиями.

Это уже хороший сценарий Human-in-the-loop.

AI выполняет роль дополнительного аналитика, но решение остаётся у команды.

16. Проверяю Definition of Done для ключевых задач

Теперь мне недостаточно просто иметь названия карточек.

Для задачи расчёта возврата я считаю работу законченной, когда правила реализованы и покрыты тестами. Должны быть проверены полный возврат, частичный возврат, превышение количества, повторный возврат уже возвращённой позиции и превышение периода в 14 дней.

Для API создания заявки я ожидаю корректную обработку валидного и невалидного запроса и проверку принадлежности заказа пользователю.

Для frontend я ожидаю не просто нарисованную форму, а возможность отправить реальную заявку в backend и корректно обработать ответ.

Для административной части должно существовать минимальное действие подтверждения и отклонения.

Для исследовательской задачи `PaymentClient.refund()` Definition of Done будет другим: здесь не требуется production-код. Мне нужен документированный технический вывод о контракте, timeout, повторных запросах и условиях безопасной реализации.

17. Итоговый Sprint Backlog

После обсуждения мой Sprint Backlog выглядит так:

Приоритет	Задача	SP	Основная зависимость
1	Модель данных возврата	3	Нет
2	Eligibility и расчёт суммы	5	Модель заказа, уточнение <code>delivered_at</code>
3	Исследование <code>PaymentClient.refund()</code>	2	Документация платёжного сервиса
4	API создания заявки	3	Модель возврата и бизнес-логика
5	API состояния возврата	2	Модель возврата
6	Web-интерфейс создания возврата	5	Согласованный API-контракт
7	Минимальная административная обработка	3	Заявка и статусы

Итого:

23 SP из доступных 24 SP

Цель спринта:

Подготовить минимальный рабочий сценарий самостоятельного создания заявки на возврат товара с расчётом суммы, административным решением по заявке и отображением её состояния пользователю.

18. Что остаётся в backlog после спринта

После этого спринта эпик ещё не обязательно завершён полностью.

Следующим этапом остаётся production-интеграция `PaymentClient.refund()`, если исследование подтвердит безопасный способ её реализации.

После неё можно завершить автоматический переход заявки в `refunded`, отправку email после возврата денег и дополнительные пользовательские улучшения.

Отдельно могут появиться решения по отмене заявки пользователем, обязательному комментарию при отклонении и поддержке старых заказов без `delivered_at`.

Таким образом, backlog не исчезает после планирования одного спринта. Мы просто определяем следующий наиболее полезный и реалистичный инкремент.

19. Итог практики

В этой практике я использовал AI-агента несколько раз, но ни разу не передавал ему право окончательно решить, что команда обязана делать.

Сначала агент мог помочь проверить границы эпика. Затем — предложить декомпозицию. После этого — дать независимую оценку сложности и найти неопределённости. В конце — выступить критиком уже готового Sprint Plan.

Само решение я сформировал на основании цели продукта, зависимостей, рисков, capacity команды и доступности специалистов.

В результате первоначальный крупный эпик превратился в управляемый Sprint Backlog объёмом 23 story points. Мы не пытались поместить весь функционал в 24 points любой ценой, а выбрали законченный пользовательский инкремент и отдельно вынесли технически неопределённую финансовую интеграцию в исследование.

Именно так я хочу использовать AI в Agile-планировании: не как автоматического Product Owner или Scrum Master, а как дополнительный аналитический инструмент, который помогает команде быстрее находить варианты, зависимости и риски, но не снимает с людей ответственность за итоговое решение.